

값에는 정확한 posterior를, 선택에는 정확한 비용을: Binary Linear System을 위한 2단계 학습 branching

Exact Posteriors for Values, Exact Costs for Choices: Two-Stage Learned Branching for Binary Linear Systems
Gwang-Jong Ko*, Taesu Cheong †, In-Chan Choi † · School of Industrial and Management Engineering, Korea
University · 2026-09-17 · 한국어판

초록

Binary linear system(BLS)은 $A \in \{0,1\}^{m \times n}$ 에 대해 $Ax=b$ 를 만족하는 $x \in \{0,1\}^n$ 를 찾는 문제다. market-split 계열 instance에서는 linear relaxation이 무정보하므로 complete search는 branching 결정에 노력을 쓴다. 본 논문은 서로 다른 질문에 답하는 두 개의 정확한 신호로 두 단계에 걸쳐 학습한 branching heuristic을 제시한다. Stage 1은 변수가 어느 값을 가져야 하는지에 답한다: 모든 해를 열거할 수 있는 작은 instance에서는 정확한 conditional marginal $\Pr[x_i=1 \mid A, b, \text{partial assignment}]$ 를 계산할 수 있고, BLS를 partial assignment로 conditioning하는 것은 그것을 축소하는 것과 동일하므로 축소 instance로 학습한 하나의 network가 tree의 모든 node에서 예측한다. marginal은 또한 node의 변수를 "잘못 분기하면 subtree가 비는 변수"와 "그렇지 않은 변수"로 분할하며 전자만이 비용을 낸다. Stage 2는 어느 변수로 분기할지에 답한다: branching 결정의 비용 — 그 subtree가 전개하는 node 수 — 은 탐색 자체가 정확히 산출하며, Stage 1 규칙에서 출발해 marginal network를 동결한 채 그 비용으로 actor-critic fine-tuning하면 branching 순서만 바뀐다. wall-clock 예산을 맞추고 3 seed로 반복한 21×60 100개 instance에서 Stage 1은 LP 기반 branching보다 2.0~2.3배 빠르고 node를 5.5배 적게 쓰며, Stage 2는 추가로 2.0~2.4배 빠르고 node를 2.3~2.4배 적게 쓴다($p < 0.001$). 학습 seed 간에 재현되고 크기 양방향으로 전이된다: 18×50 에서 fine-tuning한 policy는 21×60 에서도 규칙보다 2.0배 낮고, 두 fine-tuning policy 모두 24×70 에서 30/30을 지도 규칙보다 2.3~3.3배 빠르게 푸는데 LP 규칙은 23~25/30을 푼다; 28×80 에서는 예산 내 해결율을 두 배로 올리며(18/30 대 8/30) 거기에 유용한 차이가 닫힌다. fine-tuning된 policy는 held-out 상태의 95%에서 확신도 규칙과 다른 변수 — 대개 network가 덜 확신하는 변수 — 로 분기하며, Stage 1을 그 예측 마진이 사는 배포 크기 상태로 학습하면 tree가 두 배가 된다. branch가 맞을 확률과 틀렸을 때의 비용은 다른 목적함수이고 탐색이 최소화하는 것은 후자다. 방법이 하지 못하는 것도 보고한다: CP-SAT는 21×60 에서 여전히 9~10배 빠르고, 정확한 지도는 열거가 완료되는 곳에서만 가능하다.

키워드: Binary linear systems · Learning to branch · Exact conditional marginals · Reinforcement learning · Tree search · Graph neural networks

1. 서론

Binary linear system(BLS)은 feasibility 문제

$$\text{find } x \in \{0,1\}^n \text{ such that } Ax=b, A \in \{0,1\}^{m \times n}, b \in \mathbb{Z}^m \quad (1)$$

이며 해집합을 S 로 쓴다. 레이저 기반 비파괴검사에서 정수 투영으로부터 이진 점유 영상을 복원하는 과제로 나타나고, market-split 형태(Cornuéjols and Dawande, 1999)로는 표준적 hard 계열이다: relaxation polytope가 크고 그 vertex가 $\{0,1\}^n$ 에서 멀어 bounding이 거의 가지치지 못하므로 complete search는 branching 결정에 의해 좌우된다. lattice reduction(Aardal et al., 2000)이 많은 instance를 바로 풀고 clause learning을 갖춘 constraint solver가 나머지를 풀지만, 어느 쪽도 branching 결정 자체를 싸게 맞춰주지는 않는다.

우리는 (1)을 위한 학습 branching heuristic이 어떤 신호로 학습되어야 하는지를 묻고, branching 결정이 두 부분으로 이루어지므로 답도 두 부분임을 발견한다. 선택된 변수가 어느 값을 먼저 시도해야 하는가는 해가 어디에 있는가의 질문이고, 열거 가능한 instance에서는 정확한 답이 있다: 지금까지의 배정이 주어졌을 때 x_i 의 conditional marginal이다. 어느 변수로 분기할 것인가는 그 뒤에 따르는 subtree의 비용에 대한 질문이고, 이것도 정확한 답이 있다: 탐색이 backtrack하면서 스스로 산출한다. 첫째 신호는 확률이고 둘째는 개수다. 우리 방법은 둘 다, 이 순서로 학습하며, 본 논문의 핵심 실증 결과는 둘이 교환 불가능하다는 것이다: 가장 중요한 곳에서 확률을 개선해도 tree는 줄지 않지만 비용을 최적화하면 줄어든다, 비용 최적 policy는 확률 모델이 덜 확신하는 변수로 분기한다.

기여. (1) complete search의 모든 node에서 쓰는 정확한 conditional marginal 지도 신호. 열거한 해집합의 정확한 marginal로 network를 학습하는 것 자체는 새롭지 않다(NSNet이 SAT에서 함; Li and Si, 2022). 새로운 것은 그 target을 쓰는 자리다. (1)을 partial assignment로 conditioning하는 것은 축소하는 것과 동일하므로(명제 1) 같은 marginal이

모든 interior node에서 얻어지고, 그것이 node의 변수를 backtrack 비용을 낼 수 있는 변수(FORCED)와 그렇지 않은 변수(OPEN)로 분할한다. FORCED 비중은 root 6%에서 depth 8의 88%로 오르며, root 상태로 학습한 network는 tree 안에서 relaxation rounding보다 못하고 tree 안에서 학습한 것은 낫다. network는 안내만 하고 변수를 확정하지 않는다. (2) 탐색에서 읽어낸 선택의 정확한 비용 신호. subtree 크기를 branching policy의 결정별 return으로 쓰는 것은 확립된 것이다(Etheve et al., 2020; Scavuzzo et al., 2022); 우리 기여는 출발점과 범위다. 결정별 subtree 비용에 대한 branching 순서의 actor-critic fine-tuning은 posterior에서 유도된 규칙에서 출발해 marginal network를 동결해 순서만 바뀔 수 있게 한 채, 100개 instance에서 지도 규칙보다 node 2.3~2.4배 적고 2.0~2.4배 빠르며($p < 0.001$, 학습 seed 2개) 학습 크기 아래와 위로 전이되어 24×70 까지 2.3~3.3배 빠르며, 그 크기에서는 지도 규칙만으로는 LP 규칙을 시간에서 더는 이기지 못한다. (3) 두 신호가 다른 것을 잡는다는 증거. 잔차 측정에서 확신도 규칙이 후보 5개 중 가장 싹 것을 고르는 경우는 22%뿐이고, marginal network를 배포 크기 상태로 학습하면 마진이 사는 곳의 정확도가 1%p 오르면서 tree는 두 배가 되며, fine-tuning policy는 95%의 상태에서 확신도 규칙과 다른 선택을 한다. (4) 통제된 종단 평가: 크기 간 solution multiplicity 일치, wall-clock 예산 일치, 모든 arm에 warm start relaxation, 3 seed, 100개 test set, CP-SAT·SCIP 비교, lattice reduction을 앞세운 배포 cascade.

2. 선행연구

해집합의 marginal을 branching 신호로. 결정에 동의하는 해의 비율이 분기 기준이라는 생각은 학습 이전부터 있었다. counting-based search(Pesant et al., 2012)는 개별 제약 안에서 정확히 계산한 solution density가 최대인 변수-값으로 분기하고, belief-propagation-guided decimation(Montanari et al., 2007)은 고정할 때마다 다시 계산한 근사 marginal로 가장 편향된 변수를 고정한다. 학습 버전은 counting을 network로 대체한다. NSNet(Li and Si, 2022)은 SAT 공식의 만족 배정을 열거해 얻은 정확한 marginal로 BP 구조의 graph network를 학습하고 추정치를 반올림해 local search의 초기 배정으로 쓴다; MIP-GNN(Khalil et al., 2022), Neural Diving(Nair et al., 2020), predict-and-search(Han et al., 2023)는 수집한 준최적 해로부터 변수별 bias를 예측해 변수 고정·warm start·탐색 제한에 쓴다. 우리 Stage 1은 NSNet과 같은 target — 전체 해집합의 정확한 marginal — 을 취하되 쓰는 자리와 방식이 다르다: marginal은 conditional이며 reduction 항등식으로 complete search의 모든 node에 옮겨지고, 아무것도 확정하지 않은 채 branching을 안내하며, FORCED/OPEN 분할·ceiling 분석·in-tree 대 root 통제 비교는 우리가 아는 한 그 연구에 대응물이 없다.

Learning to branch. Khalil et al.(2016)과 Gasse et al.(2019)은 strong branching을 모방하며 후자는 bipartite graph network를 쓴다; 이득은 expert 순위의 싹 접근이고, 모방은 순위가 줄이려는 tree 크기가 아니라 순위 자체를 target으로 삼는다. tree 크기를 직접 최적화하는 것은 강화학습 연구의 한 줄기다: FMSTS(Etheve et al., 2020)는 node의 subtree 크기를 관측 가능한 Q-value로 쓰고 depth-first search에서는 모든 subtree를 최소화하면 전체 tree가 최소화됨을 보이며 from scratch로 학습한다; tree MDP(Scavuzzo et al., 2022)는 subtree 단위 credit assignment를 형식화해 tree policy gradient를 유도한다; retro branching(Parsonson et al., 2023)은 회고적으로 추출한 subtree 경로에서 학습한다; SORREL(Feng and Yang, 2025)과 PPO 기반 hybrid는 시연으로 초기화한다. Neuro#(Vaezipoor et al., 2021)은 #SAT solver의 residual formula 위의 graph network를 evolution strategies로 학습해 branching 횟수를 최소화하고 더 큰 instance로 일반화한다. 우리 Stage 2는 FMSTS/tree MDP의 credit assignment를 단순 policy gradient(Williams, 1992)로 쓴다; 더한 것은 출발점과 범위 — 정확한 marginal에서 유도된 규칙을 동결하고 branching 순서만 학습 — 이며, 그것이 개선을 순서에 귀속시키게 해주고, 학습된 순서가 가장 확신하는 변수에서 멀어진다는 발견을 가능하게 한다. 이 목적의 고전적 진술이 fail-first 원칙(Haralick and Elliott, 1980)과 impact-based search(Refalo, 2004; 배정이 탐색공간을 가장 많이 줄이는 변수 선호)다. constraint programming에서는 value ordering을 해로부터(Chu and Stuckey, 2015) 또는 graph network를 갖춘 deep Q-learning으로(Cappart et al., 2021) 학습했다; 우리는 값은 정확한 posterior에서 취하고 변수만 학습한다. 직접 satisfiability 예측은 기록이 약하고(Selsam et al., 2019; Li et al., 2023) message-passing network는 어떤 feasible/infeasible 쌍을 원리적으로 구분할 수 없다(Chen et al., 2023); 우리는 network에 판정을 요구하지 않는다.

Complete solver와 lattice 방법. branch-and-bound는 market-split instance에서 얻는 것이 적다 — 구성이 bounding이 기대는 것을 제거하기 때문이다(Cornuéjols and Dawande, 1999). clause learning을 갖춘 constraint solver(Marques-Silva and Sakallah, 1999; Ohrimenko et al., 2009)는 내려가면서 정보를 제조하며, 우리가 tree의 interior를 연구하는 이유다. Aardal–Hurkens–Lenstra embedding(Aardal et al., 2000)은 LLL/BKZ reduction(Lenstra

et al., 1982; Schnorr and Euchner, 1994)을 certificate를 내는 첫 단계로 만들며, lattice enumeration은 정식 벤치마크에서 여전히 최고 수준이다: Wassermann(2025)은 QOBLIB market-split instance를 CPU 하나로 $m=14$ 까지 푸는데 branch-and-cut은 $m=7$ 에서 멈춘다. 우리 instance는 다른 영역(크기 간 solution multiplicity 일치, $m/n \approx 0.35$)이고 우리 기여는 그 벤치마크의 기록이 아니라 branching 방법론이다; 우리 pipeline에서도 lattice reduction이 먼저 돈다.

Amortized inference. instance별 계산을 forward pass 하나로 대체하는 학습된 사상이 amortized optimization이다(Amos, 2023; Gershman and Goodman, 2014). 두 단계 모두 이 종류다: marginal network는 LP-probing(변수마다 relaxation 하나)을, cost head는 완료된 탐색만이 드러내는 subtree 비용을 상각한다.

3. 제안 방법

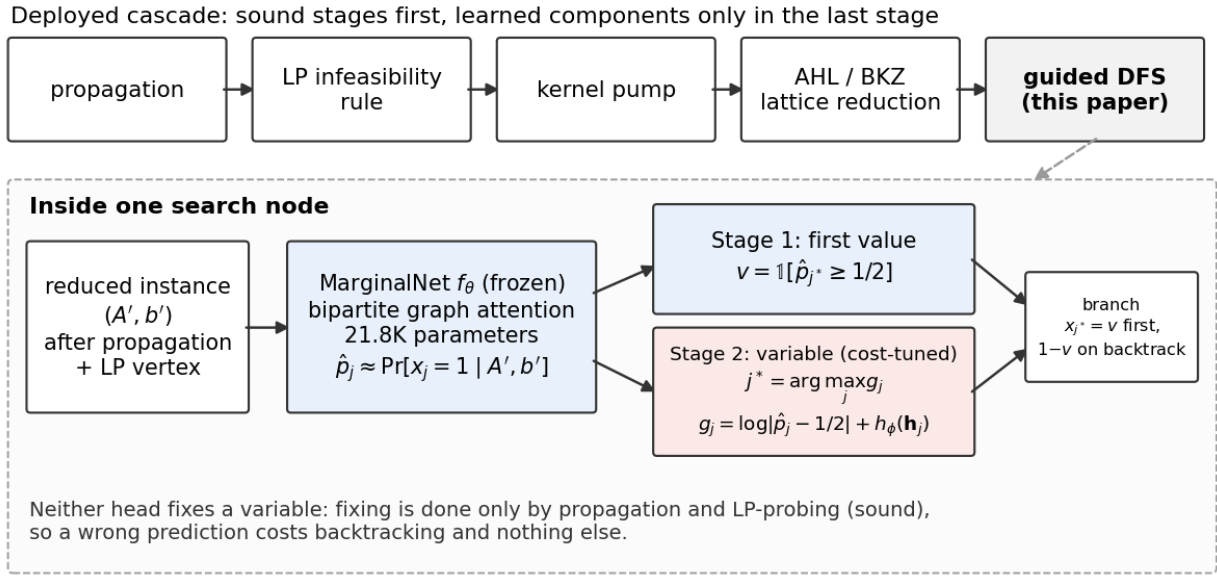


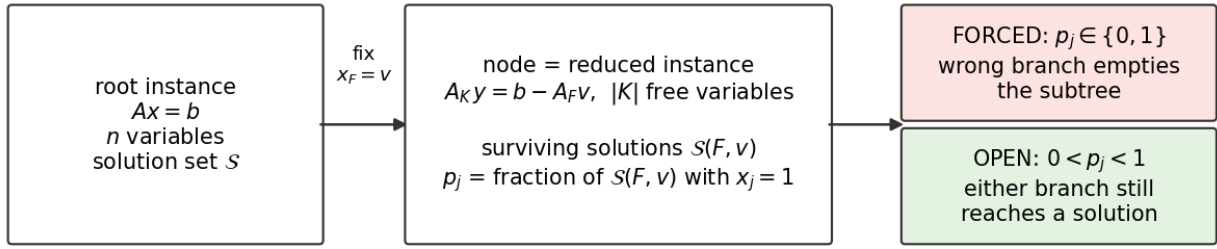
그림 1. 배포 시스템과 두 학습 컴포넌트. cascade는 건전한 단계를 먼저 돌리고 학습 컴포넌트는 마지막 guided depth-first search 안에서만 작동한다. 한 node에서 동결된 marginal network가 $\hat{p}^\wedge$ 를 내고, 먼저 시도할 값은 $\hat{p}^\wedge$ 에서(Stage 1), 분기할 변수는 비용으로 조정된 policy head에서(Stage 2) 나온다. 어느 쪽도 변수를 확정하지 않는다.

3.1 설정과 탐색

(1)이 주어지면 depth-first search는 고정 변수 집합 F 위의 partial assignment $x_F=v$ 와 자유 변수 집합 K 를 유지한다. 각 node에서 잔여 시스템에 unit propagation을 돌리고, 어떤 row가 만족 불가능해지면 가지치며, 아니면 분기한다: $j \in K$ 와 첫 값 v 를 골라 $x_j=v$ 를 먼저, backtrack 시 $x_j=1-v$ 를 탐색한다. 변수의 확정은 건전한 deduction(propagation, 반대 값이 relaxation을 infeasible하게 만들 때의 LP-probing)만이 수행한다. 학습 컴포넌트는 branching 순서와 첫 값을 정하므로 잘못된 예측의 대가는 backtracking뿐이다. 그림 1이 배포 cascade다: propagation, LP-infeasibility 규칙, kernel-pump heuristic, AHL/BKZ lattice reduction이 먼저 돌아 쉬운 부분을 닫고 guided search는 잔여를 본다.

3.2 Conditioning은 reduction이다; 어떤 결정이 비용을 낳는가

Conditioning on a partial assignment is the same as reducing the instance (Proposition 1)



Only errors on FORCED variables cost a backtrack. Their share is 6.3% at the root and 87.9% at depth 8 (10×25), so the network is trained on reduced instances (in-tree states); labels are exact because S is enumerated.

그림 2. 명제 1과 그것이 유도하는 분할. $x_F=v$ 인 node는 더 작은 BLS $A_K y = b - A_F v$ 이고 그 conditional marginal은 그 instance의 통상적 marginal이다. 살아남은 모든 해가 일치하는 변수가 FORCED이며, 그 반대로 분기할 때만 backtrack 비용이 든다.

$S(F,v) = \{x \in S: x_F=v\}$ 를 node와 일관된 해집합, $p_j = |\{x \in S(F,v): x_j=1\}| / |S(F,v)|$ 를 conditional marginal이라 하자. 이는 planted-solution 생성기가 유도하는 S 위의 균등 prior 하의 posterior다.

명제 1. A_F, A_K 를 F, K 로 인덱싱된 열 부분행렬이라 하면 $x \rightarrow x_K$ 는 $S(F,v)$ 에서 $\{y \in \{0,1\}^{|K|}: A_K y = b - A_F v\}$ 로의 전단사다. 따라서 원본 instance의 conditional marginal은 축소 instance의 통상적 marginal이다. 증명. $Ax = A_F x_F + A_K x_K$ 이고 $x_F=v$ 로 고정하면 $Ax=b \Leftrightarrow A_K x_K = b - A_F v$ 이며 x_F 가 결정되어 있으므로 전단사다. marginal 진술은 개수를 세면 따른다. $\square$

두 귀결이 있다(그림 2). 첫째, 하나의 architecture·라벨 생성기·학습 루프가 모든 node를 담당한다: depth-d 학습 상태는 n -d개 변수의 instance로 저장된다. 둘째, $p_j \in \{0,1\}$ 은 살아남은 모든 해가 x_j 에 일치한다는 뜻이고 반대로 분기하면 subtree가 빈다. 이런 변수를 FORCED, 나머지를 OPEN이라 부른다: OPEN 변수는 어느 branch로도 해에 도달하므로 첫 값이 틀려도 해를 포함한 subtree를 탐색하는 것 이상의 비용이 없다. FORCED 오류만이 backtrack 비용을 낳는다. 그 비중은 탐색이 내려가 $|S(F,v)|$ 가 줄수록 커진다 — 10×25 instance의 root에서 6.3%, depth 8에서 87.9% (§ 4.2) — 이것이 예측을 tree 안에서 학습·평가하는 이유다.

3.3 Stage 1: 정확한 conditional marginal을 target으로

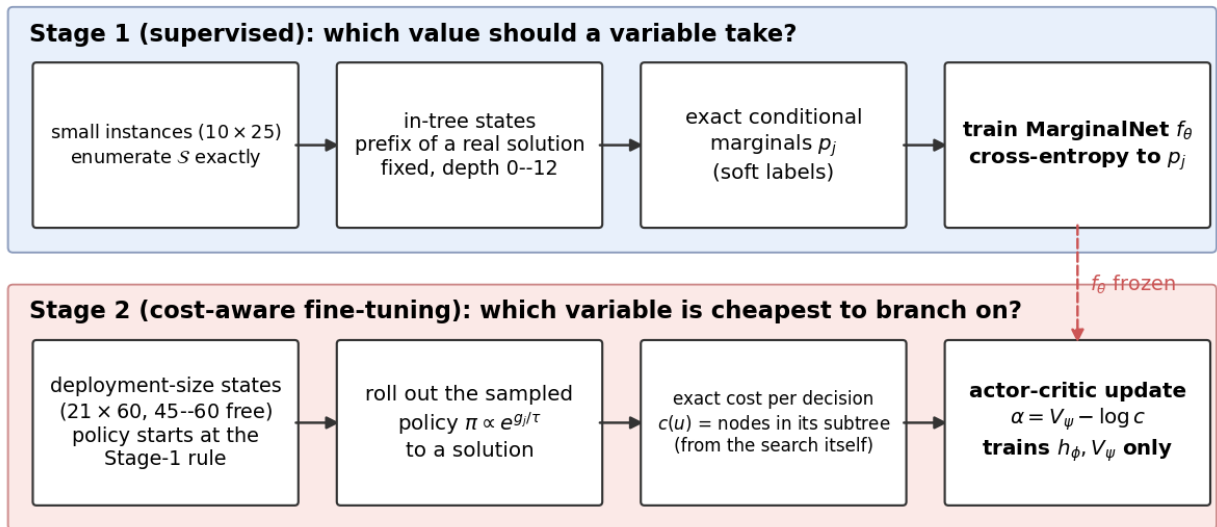


그림 3. 두 학습 단계. Stage 1은 열거 가능한 instance의 정확한 conditional marginal에서 어느 값을 배우고, Stage 2는 탐색이 산출하는 모든 결정의 정확한 subtree 비용에서 어느 변수를 배운다. Stage 1 network는 동결된다.

열거 가능한 instance(여기서는 10×25)에서 S 는 constraint solver의 all-solutions mode로 얻고 p_j 는 개수로 계산한다. 소진까지 완료된 열거(OPTIMAL 또는 INFEASIBLE)만 인정한다; 해를 이미 모은 채 시간제한에 걸린 열거는 잘린 S 를 대입해 모든 라벨을 편향시킨다. 학습 상태는 도달 가능한 in-tree 상태다: depth $d \in \{0, \dots, 12\}$ 를 균등하게 뽑고 LP 확신도 순서의 prefix를 무작위로 고른 실제 해의 값으로 고정한 뒤 축소 instance를 다시 열거해 자유 변수의 p_j 를 얻는다. network f_θ (MarginalNet)는 graph attention(Velickovic et al., 2018)을 쓰는 bipartite 변수-제약 graph network다: 변수 node당 특징 3개(LP 값, 열 밀도, fractionality), 제약 node당 3개(우변, 행 밀도, LP 잔차; 각각 행 크기로 정규화), hidden 폭 64, 양방향 attention 4 라운드(residual 연결, layer normalization), 변수당 logit 하나; 파라미터 21,761개. soft target p_j 에 대한 cross-entropy로 학습한다:

$$L_1 = -(1/|K|) \sum_{j \in K} [p_j \log p_j^\wedge + (1-p_j) \log(1-p_j^\wedge)] \quad (2)$$

이는 $p^\wedge = p$ 에서 정확히 최소화된다. p_j 자리에 심어진 해를 넣는 통상적 선택은 $|S| > 1$ 이면 잡음 섞인 대리 target이다. node에서 network는 한 번 평가되며 지도 branching 규칙은

$$j^* = \operatorname{argmax}_{j \in K} |p_j^\wedge - 1/2|, \quad v = 1[p_j^\wedge \geq 1/2] \quad (3)$$

이다.

3.4 Stage 2: 정확한 subtree 비용을 목적함수로

규칙 (3)은 첫 자식이 해를 포함할 확률을 최대화한다. 첫 자식이 해를 포함하지 않을 때 탐색이 얼마를 지불하는지, 자식이 얼마나 작아지는지는 말하지 않는다; 둘 다 subtree의 성질이고 depth-first search는 그것을 정확히 산출한다. $c(u)$ 를 node u 아래에서 해를 찾거나 subtree를 소진할 때까지 전개한 node 수라 하자; 탐색은 u 에서 backtrack해 나올 때 그것을 돌려준다. Stage 2는 $-\log c(u)$ 에 대해 branching 순서를 fine-tuning한다.

결정 과정. 상태는 propagation 후의 축소 instance, 행동은 자유 변수 j , 첫 값은 동결된 Stage 1 network의 $1[p_j^\wedge \geq 1/2]$ 로 유지한다. 각 결정이 자기 subtree를 가지므로 credit은 episode가 아니라 결정 단위로 배정된다 — sparse한 해결/미해결 보상에 없는 성질이다.

Policy와 value. f_θ 의 동결된 trunk(변수 embedding h_j) 위에 head 둘을 더한다. policy logit은

$$g_j(s) = \log |p_j^\wedge - 1/2| + h_\phi(h_j) \quad (4)$$

이며 h_ϕ 는 마지막 층을 0으로 초기화한 2층 network라서 $\pi(j|s) \propto \exp(g_j/\tau)$ 는 (3)을 부드럽게 한 것에서 출발하고 그 argmax 는 정확히 규칙 (3)이다. value head $V_\psi(s)$ 는 pooling된 graph embedding과 $\log|K|$, $\log m$ 으로 $\log c$ 를 예측한다.

갱신. 매 epoch 학습 상태를 뽑아 표집 policy($\tau=1$)를 해까지 전개하고 모든 결정을 비용과 함께 기록한다. epoch 안에서 표준화한 advantage $a(u) = V_\psi(s_u) - \log c(u)$ 로

$$L_2 = \sum_u [-a(u) \log \pi(j_u|s_u) - \lambda H(\pi(\cdot|s_u)) + (V_\psi(s_u) - \log c(u))^2], \quad \lambda=0.01 \quad (5)$$

ϕ, ψ 만 학습한다; θ 는 고정하므로 값을 고르고 FORCED/OPEN 분할을 정의하는 marginal은 변하지 않고, 어떤 개선도 branching 순서만으로 귀속된다. critic은 규칙 (3) 하에서 기록한 결정으로 사전학습하며 그 held-out 순위 상관이 actor 갱신의 관문이다(비용 순위를 못 매기는 baseline은 advantage를 잡음으로 만든다). epoch는 학습 풀의 held-out 상태로 선택하고 평가 instance는 결코 쓰지 않는다. 배포 시 policy는 greedy다: $j^* = \operatorname{argmax}_j g_j(s)$ (그림 1).

3.5 Node당 비용

자유 변수 n_c 개인 node에서 network는 relaxation 하나와 forward pass 하나가 들고, 같은 FORCED 변수를 탐지하는 건전 절차인 LP-probing은 relaxation n_c 개가 든다. 양쪽 모두 warm start relaxation으로 측정한 비율은 $n=25$ 의 1.7배에서 $n=60$ 의 7.2배, $n=150$ 의 17.8배로 커지지만, 우리 instance에서 예측이 중요한 depth에서는 1.3~1.7배에 그치며 거기서 probing은 증명까지 준다. Stage 2 head는 forward pass에 측정 가능한 비용을 더하지 않는다.

4. 실험

4.1 Instance와 프로토콜

instance는 무작위 $n/2$ -of- n 해를 심고 A마다 후보 planting 20개 중 relaxation vertex spread가 가장 큰 것을 택하는 market-split 생성기에서 나온다; 유일성은 강제하지 않는다. solution multiplicity는 과제를 바꾸므로 — $|S|=1$ 이면 모든 변수가 FORCED — 크기마다 m 을 골라 맞춘다: 10×25 ($|S|$ 중앙값 23), 18×50 (19), 21×60 (10; 27/30 검증). $n=100$ 에서는 constraint solver가 20초 안에 해를 하나도 못 찾아 모든 전략에서 탐색이 실패하므로 $n=60$ 에서 멈춘다. Stage 1은 10×25 instance 1,760개의 in-tree 상태 10,000개로 학습한다(Adam, 10^{-3} , 20 epoch, batch 1, gradient clipping 1.0; CPU 1코어 20분); hyperparameter는 튜닝하지 않았고 validation split이 없다. 평가는 크기당 30개 instance에 더해, 30개 결과가 알려진 뒤 만든 21×60 100개 세트(생성기 seed 700000~700099, 30개 포함)를 쓴다. Stage 2는 별도의 21×60 풀 500개(seed 800000+, 400/100 분할)를 쓰며 모든 평가셋과의 겹침은 (A,b) 해시 기준 0이다; 그 held-out 100개가 fine-tuning epoch 선택용 상태를 공급한다.

모든 arm은 같은 루프에서 돈다: node당 warm start relaxation 하나(Gurobi dual simplex, bound 제자리 변경), 동일 propagation, 동일 값 규칙; arm은 branching 선택만 다르다. 예산은 instance당 wall-clock이며 프로세스 종료로 강제한다; 20코어 머신에서 동시성 6; 시간에 민감한 코드는 전부 단일 스레드. instance별 속도는 양측 부호검정으로 검정한다; node 수는 relaxation vertex가 주어지면 탐색이 결정적이라 seed 불변이다. 21×60 의 모든 수치는 3 seed로 반복했다. 열거와 CP-SAT는 OR-Tools, lattice reduction은 fpylll이다.

4.2 정확한 posterior 대비 예측

	root	depth 8	전체 depth
conditional Bayes ceiling	70.8%	94.4%	87.2%
MarginalNet (in-tree 학습)	66.7%	84.3%	80.0%
MarginalNet (root 학습, 통제)	—	77.4%	75.4%
LP relaxation rounding	60.9%	80.9%	76.5%
FORCED 변수 비중	6.3%	87.9%	—

표 1. 10×25 풀의 held-out in-tree 상태에서 정확한 posterior 대비 변수별 정확도. 통제 행은 같은 풀·라벨·architecture·상태 수로 root 상태만 학습한 것이다.

표 1은 Stage 1 network를 어떤 predictor도 넘을 수 없는 ceiling, $\text{ceil} = (1/|K|) \sum_j \max(p_j, 1-p_j)$ 에 대조한다. 모든 depth에서 relaxation rounding과 ceiling 사이 격차의 대부분을 닫고, rounding 대비 3~6%p의 마진은 깊이에서 사라지지 않는다. 움직이는 것은 수준이 아니라 구성이다: FORCED 비중이 6.3%에서 87.9%로 올라 depth 8에서는 거의 모든 결정이 backtrack 비용을 낼 수 있다. 다른 모든 것을 통제하고 root 상태로 학습하면 tree 안에서 rounding보다 못한 network가 나온다(75.4% 대 76.4%): in-tree 지도가 학습된 guidance를 고전 규칙보다 낮게 만드는 요인이다. 같은 특징의 graph 없는 MLP는 동일 분할에서 79.4%, network는 84.1%라 graph 구조가 마진의 일부를 나른다. 배포 크기에서 마진이 어디 사는지는 그림 4c에 있다: 학습 범위(자유 변수 13~25) 안에서는 network와 rounding이 구별되지 않고(상관 1.000) — 제약 밀도 $m/|K| \geq 0.8$ 에서는 relaxation이 이미 모든 FORCED 변수에서 정수이기 때문 — 마진 전부(+5%p)는 학습 범위 밖인 자유 변수 50~60개, relaxation이 가장 약한 곳($m/|K| \approx 0.35 \sim 0.42$)에서 생긴다. 21×60 탐색 질의의 80%가 거기에 있다.

4.3 탐색 성능

크기 (예산)	전략	해결율	node 중앙	시간 중앙	node/s	바로 위 행 대비 (인스턴스별)
10×25 (60초)	lp	100%	46	0.03초	1,767	
	stage 1	100%	24	0.07초	376	느림: forward pass가 24-node tree보다 비쌈
18×50 (300초)	lp	100%	4,224	2.43±0.33초	2,066	
	stage 1	100%	1,085	1.82±0.37초	731	1.3~1.4배, 18/30, n.s.
	stage 1+2	100%	424	0.52±0.01초	820	1.8~2.2배, 22~24/30, p≤0.016
21×60, 30개 (600초)	lp	100%	82,366	47.9±0.8초	1,627	
	stage 1	100%	8,657	14.5±0.1초	619	2.6~3.0배, 23/30, p=0.005
	stage 1+2 (seed 0 / 1)	100%	8,873 / 4,040	12.5 / 5.9초	710 / 690	1.2~1.5배 n.s. / 2.3배, 20~21/30
21×60, 100개 (600초)	lp	100%	79,609	41.2±1.9초	1,966	
	stage 1	100%	13,386	16.5±0.6초	796	2.0~2.3배, 70~71/100, p<0.001
	stage 1+2 (seed 0 / 1)	100%	5,495 / 4,940	7.6 / 6.8초	738 / 730	2.0~2.2 / 1.9~2.4배, 64~68/100, p≤0.007

표 2. 같은 warm start 루프에서의 branching 전략, lattice 단계 off, 3 seed(seed별 중앙값의 평균±표준편차; node 수는 seed 불변). lp는 가장 정수에 가까운 relaxation 값으로 분기; stage 1은 규칙 (3); stage 1+2는 21×60에서만 학습한 비용 fine-tuning policy로 학습 seed 둘을 보고한다. 마지막 열은 각 행을 바로 위 행과 비교한다.

표 2가 주 결과다. 세 가지. 첫째, Stage 1은 모든 크기에서 LP 규칙보다 작은 tree를 만들고(node 2.0배, 3.9배, 5.5~9.5배 적음) 그 비율은 그림 4c가 예측하듯 크기와 함께 커진다; 작은 tree가 시간을 사는 것은 tree가 forward pass를 상쇄할 만큼 커진 뒤라 10×25에서는 Stage 1이 느리다. 둘째, 100개 세트는 21×60 결과를 검정력 있게 재현하되 — Stage 1이 70~71/100에서 빠름 — 마진은 원래 30개(2.6~3.0배)보다 작다(2.0~2.3배); 30개는 큰 세트의 앞 30개 seed이며 Stage 1에 유리한 표본이었고, 우리는 100개 수치를 추정치로 본다. 셋째, Stage 2는 100개에서 node를 추가로 2.3~2.4배, 시간을 2.0~2.4배 줄이며 이는 두 학습 seed와 모든 평가 seed에서 성립하고 node당 비용은 불변이라 LP 규칙 대비 종단 4.8~5.9배다; policy는 21×60에서만 fine-tuning했지만 18×50으로 전이된다(node 1.7배 적음, 1.8~2.2배 빠름). 원래 30개에서는 두 학습 seed가 갈리는데(하나는 Stage 1 근처, 하나는 1.9배 우위) 100개 세트에는 그런 산포가 없다. 참고로 random branching은 20×50에서 300초 내 0/30을 풀고 § 4.5의 21×60 잔여를 하나도 달지 못한다.

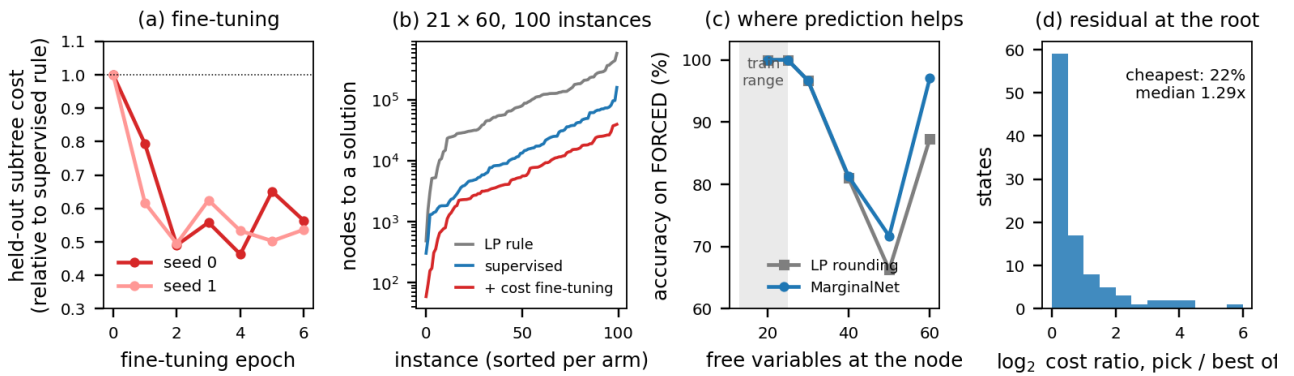


그림 4. (a) Stage 2 fine-tuning: held-out 상태 150개의 총 subtree 비용(Stage 1 규칙 대비), 학습 seed 둘. (b) 21×60 100개 세트에서 해까지의 node 수, arm별 정렬. (c) 21×60 in-tree 상태의 FORCED 변수 정확도(자유 변수 수별): Stage 1의 relaxation rounding 대비 마진은 학습 범위 밖에 있다. (d) root의 잔차: 확신도 규칙 선택의 비용을 후보 5개 중 가장 싼 것 대비로(held-out 상태 100개, 자유 변수 45~50).

4.4 Complete solver 대비

크기	CP-SAT	SCIP	stage 1	stage 1+2
10×25	0.002초	0.009초	0.07초	—
18×50	0.046초	0.385초	1.82초	0.52초
21×60 (30)	1.16초	3.69초	14.5초	5.9~12.5초
21×60 (100)	0.79초	2.59초	16.5초	6.8~7.6초
24×70	16.1초	11.7초	358~457초	125~185초
28×80	270초 (23/30)	112초	>1200초 (8/30)	720초 (18/30)

표 3. 중앙값 해결 시간, 단일 스레드, 동일 instance·예산; 전부 풀지 못한 경우 해결 수를 병기. CP-SAT는 21×60에서 전체 방법보다 9~11배, 18×50에서 11배 빠르고, 24×70부터는 SCIP이 가장 빨라 전체 방법보다 10~15배 앞선다.

이 방법은 complete solver와 경쟁하지 않는다(표 3). 격차는 분해된다: 21×60에서 CP-SAT의 branch 카운터는 중앙 18,827 branch를 보고해 우리의 4,040~8,873 node보다 많지만, 초당 19,782 node를 처리하는 반면 우리는 Python에서 약 700이며 이제 network forward pass(root에서 1.9 ms)가 node당 비용의 90% 이상이다. 부족분은 node당 처리량이지 branching 품질이 아니다; compiled inference 경로와 우리 탐색에 없는 clause learning·restart로 회복 가능한지는 보이지 않았다.

4.5 전체 pipeline에서의 배치

크기	전략	lattice	잔여	잔여 시간
10×25	모두	30/30	—	—
18×50	lp / stage 1 / 1+2	24~28/30	11/11	0.5~5.0 / 0.6~2.5 / 0.8초
21×60	random	10~16/30	0/50	>600초
21×60	lp	10~16/30	50/50	52.9~65.1초
21×60	stage 1	10~16/30	50/50	14.2~20.2초
21×60	stage 1+2	10~16/30	50/50	7.3~11.7초

표 4. lattice reduction을 앞세운 cascade, 3 seed; lattice coverage는 seed의 열 순열에 따라 변하고 한 seed 안의 모든 arm은 동일한 잔여를 받는다. 잔여 instance의 시간 중앙값(seed 범위).

lattice coverage는 크기에 따라 떨어지고(30/30, 24~28/30, 10~16/30) 그것이 branching이 중요해질 여지를 남긴다. 21×60의 잔여는 진정으로 어렵다 — random branching은 seed 합산 잔여 50개 중 하나도 닫지 못한다 — 그리고 모든 건전 전략이 전부 닫는다; 차이는 시간이다. Stage 1은 잔여에서 LP 규칙보다 3.0배 빠르고(40/50, $p=2 \times 10^{-5}$) Stage 2는 seed의 잔여에 따라 추가로 1.2~2.7배다. 종단 해결율은 모든 건전 arm 100%, random 44%다.

4.6 두 단계의 크기 transfer

arm	분기 변수	첫 값
lp	가장 정수에 가까운 relaxation 값: $\operatorname{argmax}_j x_j^{\text{LP}} - 1/2 $	반올림한 x_j^{LP}
stage 1 (25)	가장 확신하는 marginal: $\operatorname{argmax}_j p_j^\wedge - 1/2 $	$1[p_j^\wedge \geq 1/2]$
stage 1+2 (60 / 50)	비용으로 조정된 점수: $\operatorname{argmax}_j g_j$	$1[p_j^\wedge \geq 1/2]$

표 5a. branching arm. 모두 같은 depth-first 루프(node당 warm start relaxation 하나, 같은 propagation, 같은 예산)를 돌고 이 규칙만 다르다; 괄호는 학습 컴포넌트의 학습 크기. lp는 solver가 아니라 network 자리에 relaxation 값을 넣은 같은 루프다.

	CP-SAT	SCIP	lp	stage 1 (25)	stage 1+2 (60)	stage 1+2 (50)
(a) 예산 내 해결 수						
18×50 (300초)	30/30	30/30	30/30	30/30	30/30	30/30
21×60, 100개 (600초)	100/100	100/100	100/100	100/100	100/100	100/100
24×70 (1200초)	30/30	30/30	23~25/30	29/30	30/30	30/30
28×80 (1200초)	23/30	30/30	4/30	8/30	18/30	17/30
(b) wall-clock 시간 중앙값						
18×50	0.05초	0.39초	2.4초	1.8초	0.5초	1.5초
21×60, 100개	0.8초	2.6초	41초	16.5초	7.6초	10.7초
24×70	16초	12초	537~644초	358~457초	125~185초	135~165초
28×80	270초	112초	>1200초	>1200초	720초	852초
(c) 해까지의 node 중앙값 (seed 불변)						
18×50	—	—	4,224	1,085	424	1,147
21×60, 100개	—	—	79,609	13,386	5,495	7,796
24×70	—	—	980,743	260,114	90,054	95,818
28×80	—	—	1,929,041	803,535	470,606	560,077

표 5b. 두 단계의 크기 transfer, lattice off, 크기당 30개(21×60은 100개). 18×50·21×60·24×70은 seed 3개(범위), 28×80은 seed 1개. solution multiplicity는 $n \leq 60$ 에서 일치, $n \geq 70$ 에서는 열거로 검증할 수 없어 21×60의 제약 밀도($m/n \approx 0.35$)로 m을 골랐다. 괄호는 학습 크기(표 5a).

학습 → 테스트	대비	node	시간	우세
stage 1: 25→50	lp	3.9배	1.3~1.4배	18/30
stage 1: 25→60	lp	5.5배	2.0~2.3배	70~71/100
stage 1: 25→70	lp	3.8배	0.7~1.1배	16~17/30
stage 2: 60→50 (하향)	stage 1	2.6배	1.8~2.2배	22~24/30
stage 2: 50→60 (상향)	stage 1	2.0배	1.7~1.9배	63~66/100
stage 2: 60→70 (상향)	stage 1	2.7배	2.3~3.3배	22~23/30
stage 2: 50→70 (상향)	stage 1	2.2배	2.1~2.7배	19~21/30
stage 2: 50→50 (같은 크기)	stage 1	0.9배	1.1~1.4배	16~18/30

표 5c. 같은 결과를 전이 방향으로 읽은 것: node 중앙값 비율과 인스턴스별 시간 비율 중앙값(seed 범위), 전이된 arm이 더 빠른 instance 수. 24×70에서 지도 규칙은 lp 대비 node 이득은 유지하지만 시간 이득은 없고, fine-tuning policy는 둘 다 유지한다.

표 5a~5c는 모든 학습 arm을 학습 크기가 아닌 크기에서 평가한다; 표 5a는 각 arm이 무엇인지, 표 5b는 크기별 해결 수·시간·node, 표 5c는 같은 수치를 전이 방향으로 읽은 것이다. 10×25에서만 학습한 Stage 1은 모든 크기에서 LP 규칙보다 node를 적게 쓰고(3.9, 5.5, 3.8, 2.4배) |S|를 맞추면 21×60에서 18×50으로 학습한 network보다 못하지 않으며(node 8,657 대 14,742, n.s.), root 정확도는 40×100까지 rounding 대비 마진을 유지한다. 그러나 Stage 1만의 시간 이득은 tree가 커질수록 줄어든다: 24×70에서 29/30을 풀어 LP 규칙의 23~25/30보다 낮지만 둘 다 풀 instance에서는 더 빠르지 않다(0.7~1.1배) — 260,000-node tree는 260,000번의 forward pass를 지불하기 때문이다. Stage 2가 그 이득을 되살리고 위로 옮긴다. 18×50에서 fine-tuning한 policy는 21×60에서 Stage 1보다 node가 2.0배 적고(69/100, $p < 0.001$; 시간 1.7~1.9배); 두 fine-tuning policy 모두 24×70에서 모든 seed에 30/30을 풀고 Stage 1보다 2.3~3.3배(22~23/30, $p \leq 0.016$)와 2.1~2.7배(19~21/30) 빠르며 node는 2.7배·2.2배 적다 — 학습 크기의 1.2~1.4배에서다. 28×80에서는 우리가 돌린 모든 arm에 대해 창이 닫힌다: LP 규칙과 Stage 1은 1,200초 안에 30개 중 4개와 8개, fine-tuning policy는 18개와 17개를 풀고, SCIP은 중앙 112초에 30/30을 푼다. 즉 자유 변수 45~60개로 fine-tuning한 policy는 본 적 없는 크기에서 해결율을 두 배로 올리지만, complete solver가 필요 없는 곳까지 방법을 확장하지는 못한다. 두 가지를 적어둔다. 첫째, fine-tuning policy는 tree가 작을 때 자기 학습 크기에서 이득이 없다(18×50: 18×50 학습 policy가 1,147 대 1,085 node) — 그림 4c와 일관되게, 배운 것은 크고 약하게 제약된 상태에서 중요하고 그런 상태는 큰 tree에만 있다. 둘째, $n \geq 70$ 에서는 SCIP이 CP-SAT를 앞서며 둘 다 우리보다 훨씬 앞에 있다.

4.7 지도 target이 놓치는 것

학습 분포를 맞추는 것은 도움이 되지 않는다. 그림 4c는 명백한 처방을 시사한다: Stage 1을 마진이 사는 곳에서 학습하라. 21×60 학습 풀에서 자유 변수 30~60개인 in-tree 상태 2,691개를 만들어(LP 순서의 prefix를 심어진 해로 고정; 정확한 marginal; 자유 변수 ≤ 55 인 상태는 전부 90초 내 열거, root는 291/400) 같은 architecture를 그것만으로(M1), 반반 혼합으로(M2) 학습했다. 자유 변수 50~60에서 FORCED 정확도는 약 1%p 오른다(Stage 1의 +5.4/+9.7 대 +6.1/+10.7). tree는 두 배가 된다: M1은 100개 세트에서 node 중앙 21,337개를 전개해 13,386보다 많고(더 적은 경우 38/100, $p=0.021$), M2는 18,558, 자유 변수 ≥ 50 인 node를 M1로 보내는 switch는 아무것도 회복하지 못한다(17,252). 이유는 node가 있는 곳이다: 전체 결정의 2/3가 자유 변수 25~34개에서 일어나며, 그런 작고 촘촘한 상태에서 M1의 최고 확신 변수가 FORCED이면서 맞는 경우는 77.6%로 Stage 1의 94.4%보다 낮다(실제 마진과의 순위 상관 0.12 대 0.36). tree 상단에서 조금 낮고 내부에서 훨씬 못한 network는 탐색을 느리게 한다. 10×25 안에서 depth 0~6, 6~12, 0~12로 학습해도 차이가 없다(79.3/79.9/79.9%). 학습 상태 분포는 지렛대가 아니다.

확신하는 선택은 드물게만 가장 싸다. 자유 변수 45~50개 held-out 상태 100개에서 root를 가장 확신하는 변수 5개 각각으로 강제하고 아래는 규칙 (3)으로 돌렸다. 규칙 자신의 선택이 5개 중 가장 싼 경우는 22%; 최선 후보 대비 비용 비율 중앙값 1.29, 평균 2.97; 2배 이상 싼 후보가 있는 상태 24%; root 최선 선택만으로도 이 상태들의 총 node 55%가 제거된다(그림 4d). 잔차는 어느 변수인가에 관한 것이고 크다.

4.8 비용 기반 fine-tuning

Critic 관문. 학습 풀에서 규칙 (3)의 모든 결정을 기록하면 결정 37,275개(held-out 9,375개)와 정확한 subtree 비용이 나온다. 동결 trunk 위의 value head는 log c와 held-out Spearman 상관 0.80에 이르고, 상태 크기만 아는 baseline은 0.70이며 크기 구간 안에서는 critic의 우위가 더 크다(자유 변수 35~44에서 0.68 대 0.38). trunk를 풀면 0.02가 더해진다; 동결 critic을 쓴다.

Fine-tuning. 자유 변수 45~60개 학습 상태에서 rollout 400개씩 6 epoch($\tau=1$, epoch당 약 21,000 결정). 그림 4a가 held-out monitor다: 총 subtree 비용이 Stage 1 규칙의 0.46(seed 0, epoch 4 선택)과 0.50(seed 1, epoch 2)으로 떨어지고 policy entropy는 1.7에서 0.1 아래로 떨어진다. 평가는 표 2와 그림 4b에 있다.

무엇이 바뀌었나. held-out 상태 300개에서 fine-tuning policy는 95%에서 규칙 (3)과 다른 root 변수로 분기한다; 그 선택은 대개 15번째로 확신하는 변수이고($|p^*-1/2|$ 0.26 대 0.43) FORCED이면서 첫 값이 맞을 확률도 낮다(57.5% 대 82.5%). policy는 network가 가장 확신하는 변수에서 멀어졌고 탐색은 싸졌다. 이득은 root 결정만의 성질이 아니다: root만 fine-tuning 선택으로 강제하고 아래를 규칙 (3)으로 돌리면 비용 비율 중앙값 0.93(22/46에서 더 싼)이라, 개선되는 것은 결정 수열이다. 두 학습 seed는 많은 node에서 다른 변수를 고르면서 비슷한 비용에 도달하는데, 특정 해가 아니라 목적함수가 학습됐다면 기대되는 바다. fine-tuning이 무엇을 최적화하는지는 말할 수 있어도 그것이 이용하는 특징의 이름은 아직 붙이지 못했다; 값싼 대리 지표인 첫 자식의 propagation 양은 두 선택 사이에 차이가 없다.

5. 한계

정확한 지도가 방법을 제한한다. marginal은 S의 열거를 요구하고 이는 $n \approx 60$ 을 넘으면 완료되지 않는다; Stage 1은 큰 크기로 전이되지만 거기서 학습할 수 없고, Stage 2의 비용 신호는 탐색이 도는 어느 크기에서도 얻을 수 있지만 21×60 에서만 썼다. 유용한 크기 창은 양쪽으로 막혀 있다. 아래에서는 lattice reduction이 전부를 닫고 위에서는 탐색이 실패한다: 28×80 에서 fine-tuning policy는 1,200초 안에 18/30, Stage 1만으로는 8/30을 풀고 $n=100$ 에서는 어떤 전략도 해를 찾지 못한다; 시연은 $n=50 \sim 70$ 에 있다. $n \geq 70$ 에서는 열거로 multiplicity를 검증할 수 없어 제약 밀도만 맞췄고 28×80 은 seed 1개다. 시스템은 CP-SAT와 경쟁하지 못한다(9~11배 느림). 부족분은 forward pass가 지배하는 Python 루프의 node당 처리량이며, compiled 경로와 clause learning이 이를 닫는지는 보이지 않았다. 검정력과 표집. 원래의 21×60 30개는 Stage 1에 유리하고 Stage 2에는 불안정한 표본이었다; 우리가 지지하는 추정치는 100개 세트이고 크기 간 결과는 크기당 30개에 기댄다. Stage 2는 학습 풀 둘과 seed 둘에 기댄다. 학습 seed 둘이 21×60 에서 일치하고 이득은 $18 \times 50 \cdot 24 \times 70$ 으로, 해결율로는 28×80 으로 전이되지만 $n=60$ 위에서 학습한 policy는 없다. entropy가 4 epoch 안에 붕괴하고 epoch는 중단 규칙이 아니라 monitor로 고르며 이용하는 특징의 이름이 없다. 어느 단계의 hyperparameter도 튜닝하지 않았다; Stage 1의 depth 범위는 뒤에 한 크기 안에서 중요하지 않음이 확인됐다. 단일 instance 계열. 어느 신호든 다른 constraint 계열로 전이되는지는 미검증이다. 무엇이 새롭고 무엇이 아닌가. 지도 target으로서의 정확한

marginal(NSNet)과 결정별 return으로서의 subtree 크기(FMSTS, tree MDP)는 모두 기존 것이다; 방법의 novelty는 전자를 complete search 안에서 conditional·guidance 전용으로 쓴 것과, posterior에서 유도한 동결 출발점에서 후자로 branching 순서만 fine-tuning한 것에 있다. 두 신호가 다른 것을 잡는다는 실증 주장은 본 논문의 것이며 하나의 instance 계열에 기댄다.

6. 결론

branching 결정은 두 질문을 던지고, 우리는 각각의 정확한 답으로 학습했다. 정확한 conditional posterior — 해를 열거할 수 있는 곳에서 계산되고 reduction 항등식으로 모든 node에 옮겨지는 — 는 탐색에 어느 값을 시도할지 말해주고 어떤 예측이 비용을 낳는지 특정한다; 정확한 subtree 비용 — 탐색이 backtrack하며 산출하는 — 은 어느 변수로 분기할지 말해준다. 첫째 신호는 21×60 hard instance 100개에서 LP 기반 branching을 2.0~2.3배 앞서고, 둘째는 첫째를 건드리지 않는 fine-tuning으로 2.0~2.4배를 더하며 seed 간에 재현되고 크기 양방향으로 전이된다 — 24×70 에서는 그것이 학습된 탐색을 LP 규칙보다 앞서게 하는 유일한 요소다. fine-tuning policy가 posterior가 덜 확신하는 변수를 선호한다는 것과, posterior를 그 마진이 사는 곳에서 버리면 tree가 커진다는 것은 같은 사실을 두 번 본 것이다: 맞을 확률과 틀렸을 때의 비용은 다른 목적함수이고, 탐색이 최소화하는 것은 후자뿐이다.

재현성

코드는 proposed_src/ 아래에 디렉토리별 ROLES.txt와 함께 정리되어 있다. Stage 1: v15_bayes/v15_gen_by_m.py, v22_transfer/v22_split.py, v17_conditional/v17_make_conditional_data.py, v17_train_conditional.py. 탐색·평가: v23_ablation/run_revision2.sh. Stage 2와 잔차 실험: v24_deploy_range/run_v24.sh, run_v24_r1.sh, run_v24_r2.sh, run_v24_r2b.sh; 그림: make_260917_figures.py. 모든 생성기는 seed가 고정되어 있고 instance 데이터와 가중치는 배포 대신 재생성한다.

참고문헌

- Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.
- Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.
- Amos, B. (2023). Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732.
- Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*.
- Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *Integer Programming and Combinatorial Optimization*, pages 284–293.
- Etheve, M., Alès, Z., Bissuel, C., Juan, O., and Kedad-Sidhoum, S. (2020). Reinforcement learning for variable selection in a branch and bound algorithm. In *CPAIOR*, pages 176–185.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 32.
- Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 36.
- Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. (2023). A GNN-guided predict-and-search framework for mixed-integer linear programming. In *International Conference on Learning Representations*.
- Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313.
- Khalil, E. B., Morris, C., and Lodi, A. (2022). MIP-GNN: A data-driven framework for guiding combinatorial solvers. In *AAAI Conference on Artificial Intelligence*.
- Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In *AAAI Conference on Artificial Intelligence*.
- Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.

- Li, Z. and Si, X. (2022). NSNet: A general neural probabilistic framework for satisfiability problems. In *Advances in Neural Information Processing Systems*, volume 35.
- Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*.
- Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.
- Montanari, A., Ricci-Tersenghi, F., and Semerjian, G. (2007). Solving constraint satisfaction problems through belief propagation-guided decimation. In *Allerton Conference on Communication, Control, and Computing*.
- Nair, V., Bartunov, S., Gimeno, F., et al. (2020). Solving mixed integer programs using neural networks. *arXiv:2012.13349*.
- Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.
- Parsonson, C. W. F., Laterre, A., and Barrett, T. D. (2023). Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In *AAAI Conference on Artificial Intelligence*.
- Pesant, G., Quimper, C.-G., and Zanarini, A. (2012). Counting-based search: Branching heuristics for constraint satisfaction problems. *Journal of Artificial Intelligence Research*, 43:173–210.
- Refalo, P. (2004). Impact-based search strategies for constraint programming. In *Principles and Practice of Constraint Programming (CP)*, pages 557–571.
- Scavuzzo, L., Chen, F. Y., Chételat, D., Gasse, M., Lodi, A., Yorke-Smith, N., and Aardal, K. (2022). Learning to branch with tree MDPs. In *Advances in Neural Information Processing Systems*, volume 35.
- Cappart, Q., Moisan, T., Rousseau, L.-M., Prémont-Schwarz, I., and Cire, A. A. (2021). Combining reinforcement learning and constraint programming for combinatorial optimization. In *AAAI Conference on Artificial Intelligence*.
- Chu, G. and Stuckey, P. J. (2015). Learning value heuristics for constraint programming. In *CPAIOR*.
- Feng, S. and Yang, Y. (2025). SORREL: Suboptimal-demonstration-guided reinforcement learning for learning to branch. In *AAAI Conference on Artificial Intelligence*.
- Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R. B., Seshia, S. A., and Bacchus, F. (2021). Learning branching heuristics for propositional model counting. In *AAAI Conference on Artificial Intelligence*.
- Wassermann, A. (2025). Solving the market split problem with lattice enumeration. *arXiv:2508.08702*; *Mathematical Programming Computation*, to appear.
- Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66:181–199.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations*.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.